LAB ASSIGNMENT 2

Server Side Web Development with Node.js(CSE 3192)

Semester: 6th

1. Write a function that accepts a number and a callback as the arguments. The function will return an EventEmitter that emits an event called tick every 500 milliseconds until the number of milliseconds is passed from the invocation of the function. The function will also call the callback when the number of milliseconds has passed, providing, as the result, the totalcount of tick events emitted. (Hint: you can use setTimeout() to schedule another setTimeout() recursively)
2. Modify the function created in Question 1 so that it emits a tick event immediately after the function is invoked.
3. Modify the function created in Question 2 so that it produces an error if the timestamp at the moment of a tick is divisible by 5. Propagate the error using both the callback and the event emitter. (Hint: use Date.now() to get the timestamp and the remainder (%) operator to check whether the timestamp is divisible by 5.)
4. Analyze the design and implementation of a callback-based concatFiles() function that accepts multiple source file paths and a destination file. Examine how the function handles a variable number of arguments (e.g., using rest parameters), and evaluate how asynchronous file operations are coordinated to ensure that the contents of the source files are concatenated in the correct order without causing race conditions or data inconsistency.
5. Write listNestedFiles(), a callback-style function that takes, as the input, the path to a directory in the local filesystem and that asynchronously iterates over all the subdirectories to eventually return a list of all the files discovered. Here is what the signature of the function should look like:
   function listNestedFiles (dir, cb) { /* ... */ }
6. Write recursiveFind(), a callback-style function that takes a path to a directory in the local filesystem and a keyword, as per the following signature:
   function recursiveFind(dir, keyword, cb) { /* ... */ }. The function must find all the text files within the given directory that contain the given keyword in the file contents. The list of matching files should be returned using the callback when the search is completed. If no matching file is found, the callback must be invoked with an empty array. As an example test case, if you have the files foo.txt, bar.txt, and baz.txt in myDir and the keyword 'batman' is contained in the files foo.txt and baz. txt, you should be able to run the following code: recursiveFind('myDir', 'batman', console.log)
   // should print ['foo.txt', 'baz.txt']
   Bonus points if you make the search recursive (it looks for text files in a subdirectory as well). Extra bonus points if you manage to perform the search within different files and subdirectories in parallel, but be careful to keep the number of parallel tasks under control!
7. Implement a custom version of Promise.all() using promises, async/await, or a combination of both. Ensure that your implementation behaves functionally the same as the original Promise.all() method, handling multiple promises and returning their resolved values correctly.

8. Refactor the internal implementation of the TaskQueue class by replacing promise-based logic with async/await wherever possible. Identify and explain the parts where async/await cannot be applied.
9. Update the TaskQueuePC class internal methods so that they use just promises, removing any use of the async/await syntax. Hint: the infinite loop must become an asynchronous recursion. Beware of the recursive Promise resolution memory leak!
10. Implement a function mapAsync(iterable, callback, concurrency) that performs a parallel asynchronous mapping operation similar to Array.map(). The function should process items concurrently up to a specified limit, support both promises and synchronous return values from the callback, and ensure that the results maintain the original order of the iterable without directly using TaskQueue or TaskQueuePC.
11. Write a command-line script that takes a file as input and compresses it using the different algorithms available in the zlib module (Brotli, Deflate, Gzip). You want to produce a summary table that compares the algorithm's compression time and compression efficiency on the given file.
12. On Kaggle, you can find a lot of interesting data sets, such as the London Crime Data (nodejsdp.link/london-crime). You can download the data in CSV format and build a stream processing script that analyzes the data and tries to answer the following questions:
    (a) Did the number of crimes go up or down over the years?
    (b) What are the most dangerous areas of London?
    (c) What is the most common crime per area?
    (d) What is the least common crime?
    (Hint: You can use a combination of Transform streams and PassThrough streams to parse and observe the data as it is flowing. Then, you can build in memory aggregations for the data, which can help you answer the preceding questions. Also, you don't need to do everything in one pipeline; you could build very specialized pipelines (for example, one per question) and use the
    fork pattern to distribute the parsed data across them.

13. Analyze the design of a TCP-based client-server system for file transfer, and examine how encryption and multiplexing/demultiplexing techniques can be used to securely support simultaneous transfer of multiple files.
14. Analyze how Readable streams can be used to generate and control real-time terminal animations similar to the output of curl parrot.live..